Recursion

Tanvir Ahmed, PhD

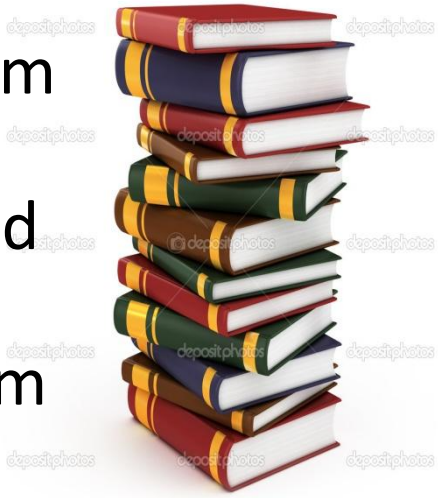
Before starting recursion

- Before starting to learn recursion, you need to understand two concepts:
- - General Stack data structure
- - Function call stack

Brief idea of stack



- A **stack** is a Last-In, First-Out (LIFO) data structure
- We have a whole topic on stacks (but this slide is just a summary of it)
- Anything added goes on the top
- Anything removed from the stack is taken from the “top”
- Things are removed in the reverse order from that in which they were inserted
- Putting an item on the top of a stack is called **PUSH**.
- Removing and getting the topmost item from the stack is called **POP**



Function call Stack

- Key idea:
  - Whenever a function call is made, the code needs to remember where to go back after completing the function's job of the function
  - So, it pushes some key data; for simplicity, just assume the return line number so that it remembers where to come back

Example function call stack

```
1 #include <stdio.h>

2 void funcC() {
3   printf("Inside C\n");
4 }

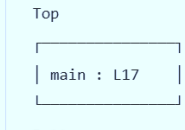
5 void funcB() {
6   printf("Start B\n");
7   funcC(); // Call at line 7
8   printf("End B\n");
9 }

10 void funcA() {
11   printf("Start A\n");
12   funcB(); // Call at line 12
13   printf("End A\n");
14 }

15 int main() {
16   printf("Program Start\n");
17   funcA(); // Call at line 17
18   printf("Back in main\n");
19   printf("Program End\n");
20   return 0;
21 }
```

Step 1: main executes line 17

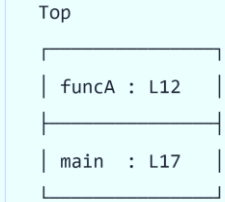
Call Stack



Execution jumps to `funcA()`.

Step 2: funcA executes line 12

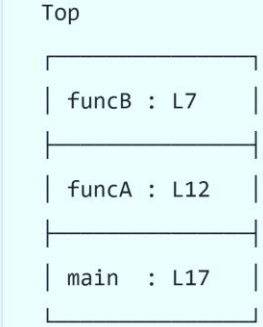
Push return address (line 12).



Execution jumps to `funcB()`.

Step 3: funcB executes line 7

Push return address (line 7).



Execution jumps to `funcC()`.

Step 4: funcC completes

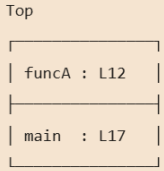
Pop stack.

```
POP → funcB : L7
```

Control returns to:

```
1  8 printf("End B\n");
```

Current stack:



Step 5: funcB completes

Pop stack.

```
POP → funcA : L12
```

Control returns to:

```
1  13 printf("End A\n");
```

Current stack:

Step 6: funcA completes

Pop stack.

```
POP → main : L17
```

Control returns to:

```
1  18 printf("Back in main\n");
2  19 printf("Program End\n");
```

Stack becomes empty.

```
Call Stack Empty
```

Recursion

- A powerful problem solving strategy
- Definition:
  - If the body of a function call the function itself.
- As you can call a function *fun1()* from a function *fun2(),* you are also allowed to call *fun1()* from *fun1().*
- What can be the potential problem?:
  - Let's say you are calling *fun1* from the body of *fun1*
  - *Wouldn't it be like an infinite loop?*
  - *How can it stop?*
  - *So, inside the body of your function you must put something that must prevent it to call the function again (i.e., must not result in a recursive call to prevent to fall in infinite loop).*

This is how I learned recursion algorithm:



Example1

```
void rec2(int x)
{
   if (x==0)
      return; //this breaking condition will result in poping the function call stack

   rec2(x-1); //calling itself (recursion)

   printf("%d  ", x); //this line (including the value of x)  is kept in the stack for each call of rec(x-1) in the above line
}
```

```
void rec1(int x)
{
   if (x==0)
      return; //breaking condition

   printf("%d  ", x); //prints first then recursion
   rec1(x-1); //calling itself(recursion)
}
```

```
int main()
{
   printf("Calling rec1: ");
   rec1(5);

   printf("\nCalling rec2: ");
   rec2(5);
}
```

```
Output:
Calling rec1: 5  4  3  2  1
Calling rec2: 1  2  3  4  5
```

Analyze rec1 of Example 1

```
void rec1(int x)
{
   if (x==0)
      return; //breaking condition

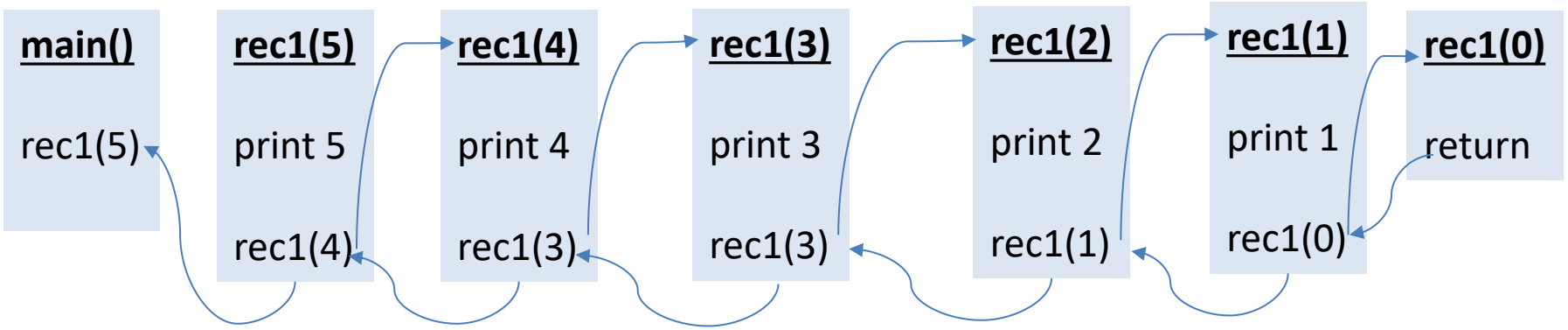
   printf("%d  ", x); //prints first then recursion
   rec1(x-1);
}
```

- It is a very simple example of recursion.
- The printf prints the value of x at a particular call and then a recursive call is made with x-1, which means the value of x is decreased by one for the next call.
- As soon as the value of x is 0, the function returns to the most recent call and prevents it from making the recursive call again.
- As there is no statement after the recursive call, noting to be kept to execute later.
- call: rec1(5) from main function.
- printf("%d ", 5); call: rec1(4)
- printf("%d ", 4); call: rec1(3)
- printf("%d ", 3); call: rec1(2)
- printf("%d ", 2); call: rec1(1)
- printf("%d ", 1); call: rec1(0)
- Return to the previous call and so on. Actually, the function completes its full task after returning to all previous calls. But nothing was pending for each call in this example
- So, the output is: 5 4 3 2 1
- Why is 0 not in the output?

Visualization of processing rec1

```
void rec1(int x)
{
   if (x==0)
      return; //breaking condition

   printf("%d  ", x); //prints first then recursion
   rec1(x-1); //calling itself(recursion)
}
```



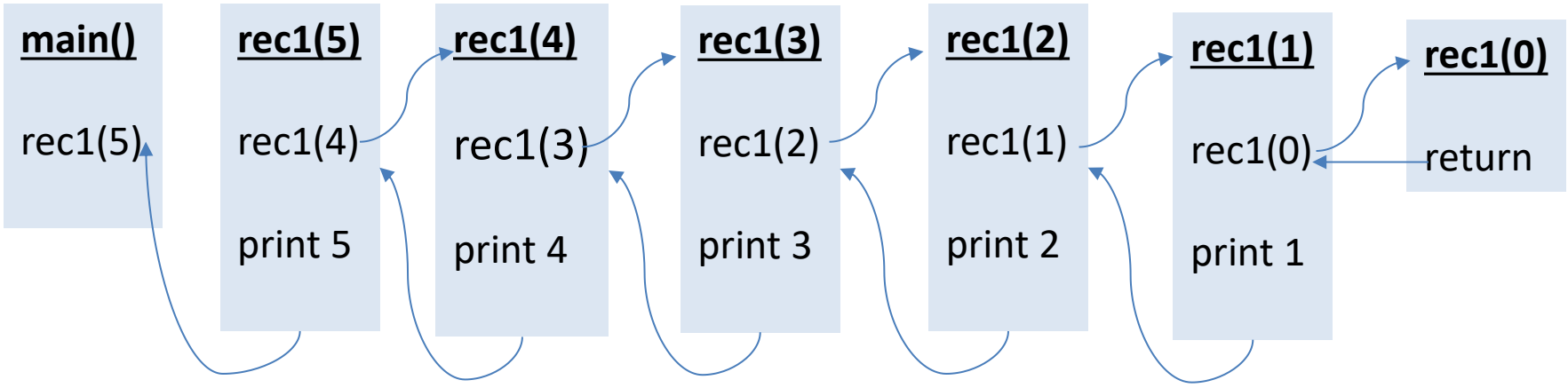
Analyze rec2 of Example 1

```
void rec2(int x)
{
   if (x==0)
      return;  // it is the breaking condition of the recursion call.
    rec2(x-1);
   printf("%d  ", x);
}
```

- rec2(x-1) is preventing the printf to work while performing the operation of a particular call.
  - ***So, the lines after rec2(x-1); are waiting to be processed after coming back***
- call: rec2(5) ->push and wait printf("%d ", 5);
- call: rec2(4) ->push and wait printf("%d ", 4);
- call: rec2(3) ->push and wait printf("%d ", 3);
- call: rec2(2) ->push and wait printf("%d ", 2);
- call: rec2(1) ->push and wait printf("%d ", 1);
- call: rec2(0) -> return and keep popping.
- So, the output is: 1 2 3 4 5
- Why 0 is not in the output?

Visualization of processing rec2

```
void rec2(int x)
{
   if (x==0)
      return;  // it is the base case of the recursion call.
    rec2(x-1);
   printf("%d  ", x);
}
```



Explaining Example 1

- Can't you print the same numbers using a loop?

```
Function rec1:
for(i=10; i>0; i--)
    printf("%d" , i);
```

```
Function rec2:
for(i=1; i<=10; i++)
    printf("%d" , i);
```

Loop vs Recursion

- **Anything computable with loops can also be computed using recursion, and vice versa.**
- **Direct Comparison: Recursion vs. Iteration**

Feature	Recursion	Iteration (Loops)
Control Flow	Repeated function calls.	Repeated block execution.
Termination	Reaching a defined **base case**.	Falsifying a **loop condition**.
Memory Overhead	High (allocates a new stack frame per call).	Low (uses a fixed amount of memory).
Risk	**Stack Overflow** if recursion is too deep.	Infinite loop (program hangs but doesn't crash memory).
Readability	cleaner for hierarchical data (trees, graphs).	Cleaner for linear sequences (arrays, lists).

Example2: Power calculation

- Calculating power $\mathbf{b^e}$
- It means you need to multiply b (e times). b*b*b*… e times
- Loop based iterative solution:

```
int Power(int base, int exponent){
        long result = 1;

        while (exponent != 0)
        {
                result *= base;
                --exponent;
        }
        return result;
}
```

- How can you use recursion to do it?

Example2

```
// Pre-conditions: e is greater than or equal to 0.
// Post-conditions: returns b^e.
int Power(int base, int exponent) {

    if ( exponent == 0 )
        return 1;
    else
        return (base*Power(base, exponent-1));
}
```

Say we were trying to evaluate the expression:

- **Power(**5, 2)
- **Power(**5,2) returns 5***Power(**5,2-1) = 5***Power(**5,1**)**
- **To evaluate this expression we must evaluate:**
- **Power(**5,1**)** returns 5***Power(**5, 0**)**
- **Finally, we have:**
- **Power(**5,0**) returns 1.**
- **Thus, Power(**5,1**) returns 5*1, which is 5.**
- **Finally, Power(**5,2) returns 5***Power(**5,2-1) = 5*5 = 25,
- **and we are done.**

General structure of a Recursive Functions

- We can see that in recursion in general:
  - when we have a problem, we want to break it down into chunks, where one of the chunks is a smaller version of the same problem.
- In the case of Power, we utilized the fact that
  - $x^y = x * x^{y-1}$, and realized that x^{y-1} is in essence an easier version of the original problem.
- We break down our original problem enough that our sub-problem is quite easy to solve.
  - At this point, rather than making another recursive call, directly return the answer, or complete the task at hand.
- So, ideally, a general structure of a recursive function has a couple options:
  - Break down the problem further, into a smaller subproblem
  - OR, the problem is small enough on its own, solve it.
- When we have two options, we often use an if statement. This is typically what is done with recursion.

General structure of a Recursive Functions

- **Here are the two general constructs of recursive functions:**

Construct 1

```
if (terminating condition) {
        DO FINAL ACTION
}
else {
        Take one step closer to terminating condition
        Call function RECURSIVELY on smaller subproblem
}
```

Construct 2

```
if (!(terminating condition) ) {
        Take a step closer to terminating condition
        Call function RECURSIVELY on smaller subproblem
}
```

- Typically, functions that return values take on the first construct,
- while void recursive functions use the second construct.
- Note that these are not the ONLY layouts of recursive programs, just common ones.

Example using construct 1

- Let's write a function that takes in one positive integer parameter n, and returns the sum 1+2+...+n. (You probably did this is C)
- Can you write the iterative solution that you have done in C?

```
// Pre-condition: n is a positive integer.
// Post-condition: Function returns the sum 1+2+...+n
int Triangle_Number(int n) {
        int index, sum = 0;
        for (index=1; index <=n; index++)
                sum = sum + index;

    return sum;
}
```

Example using construct 1

- Let's write the recursive solution of the example:
- Step 1: We need to solve this problem is such a way that part of the solution is a sub-problem of the exact same nature.
- Let *f(n)* denote the value *1+2+3+...+n*
- Using our usual iterative logic, we have:

  $$f(n) = 1 + (2 + 3 + \ldots + n)$$

- But, *2+3+... + n* IS NOT a sub-problem of the form *1+2+...+n*.
- But, let's try this:

  $$f(n) = 1 + 2 + \ldots + n = n + (1 + 2 + \ldots + (n-1))$$

  $$f(10) = 1 + 2 + \ldots + 10 = 10 + (1 + 2 + \ldots + 9) = 10 + f(9)$$

- So, here we have gotten the expression *1 + 2 + ... + (n-1)* which IS a sub-problem we were looking for.
- Hence, we have

  $$f(n) = 1 + 2 + \ldots + n = n + (1 + 2 + \ldots + (n-1)) = n + f(n-1)$$

Example using construct 1

- If we look at the construct 1, the first thing we need to determine is a terminating condition.
  - We know that f(0) = 0, so our terminating condition can be n=0.
- Furthermore, our recursive call needs to be returning an expression for f(n) in terms of f(k), for some k<n.
  - In this case, we just found that f(n) = n + f(n-1). So, now we can write our function: can you write it?

```
// Pre-condition: n is a positive integer.
// Post-condition: Function returns the sum 1+2+...+n
int Triangle_Number(int n) {

    if ( n == 0 )
        return 0;
    else
        return (n + Triangle_Number(n-1));
}
```

Two Other Classic Examples that Use Construct #1

- For positive integers n, n! (read, "n factorial"), is defined as follows:

  n! = 1x2x3x4x…xn

- What is a factorial?
  - 4! = 4 * 3 * 2 * 1 = 24
  - In general, we can say:
  - n! = n * (n-1) * (n-2) * … * 2 * 1
    - Also, 0! = 1(just accept it!)
- Mathematically, factorial is already defined recursively (**Note that each factorial is related to a factorial of the next smaller integer)**
- 4! = 4*3*2*1 = 4 * (4-1)! = 4 * (3!)
- Another example:
  - 10! = 10*9*8*7*6*5*4*3*2*1
  - 10! = 10 * (9!)
- So:
  - n! = n*(n-1)!
  - What would be the stopping condition?
  - We let 0! = 1;
- Why don't you write the recursive function?

Two Other Classic Examples that Use Construct #1

- So, the recursive code for factorial would look like this:

```
#include <stdio.h>
void Fact(int n);
int main(void)
{
        int factorial = Fact(10);
        printf("%d\n", factorial);
        return 0;
}

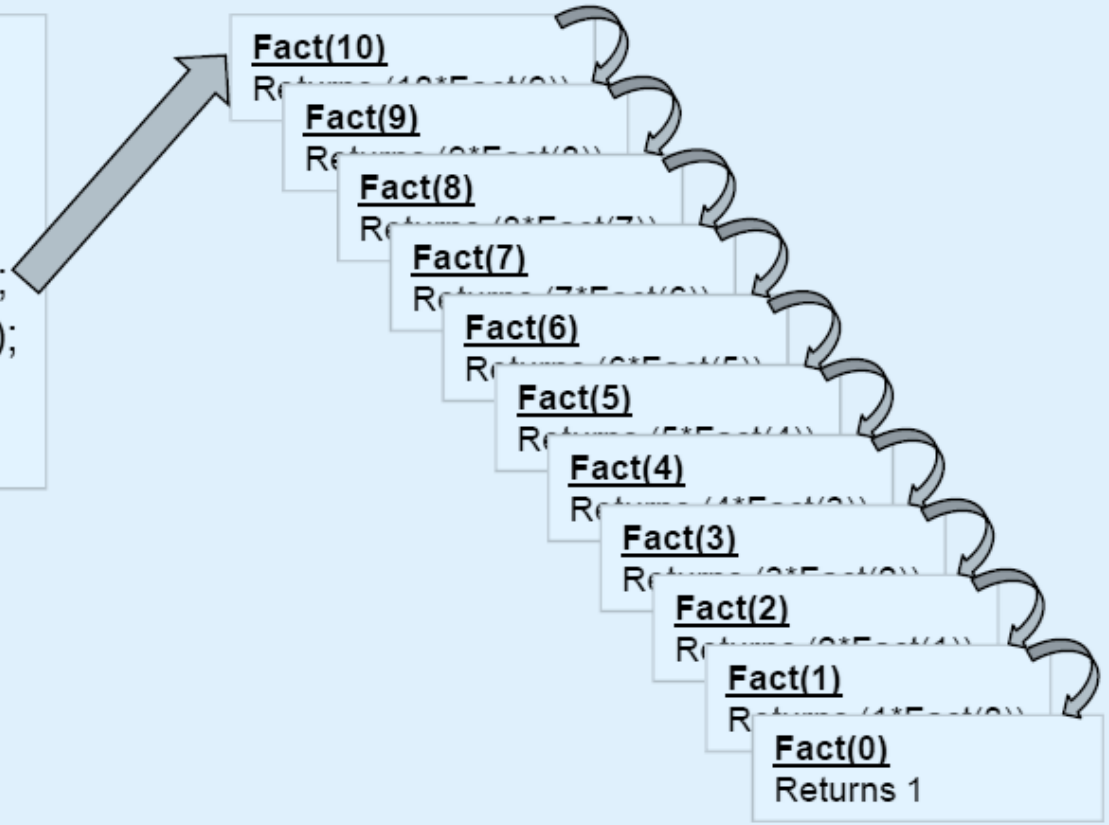

//Here's the Fact Function
int Fact (int n)
{
        if (n == 0)     return 1;
        else
            return (n * Fact(n-1));
}
```

This program prints the result of 10*9*8*7*6*5*4*3*2*1
Which is:3628800

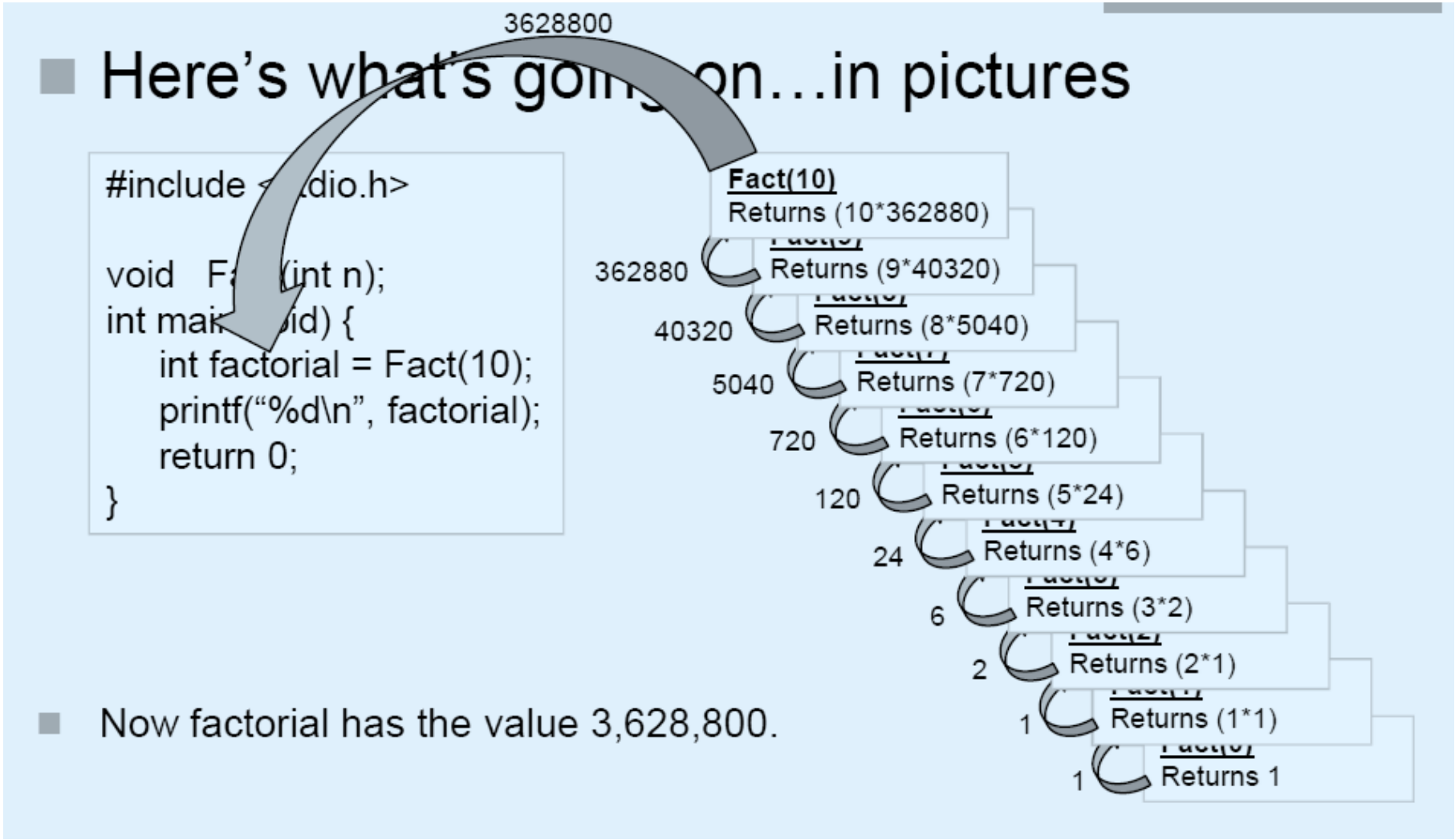
Here's what's going on…in pictures

```
#include <stdio.h>

void   Fact(int n);
int main(void) {
     int factorial = Fact(10);
     printf("%d\n", factorial);
     return 0;
}
```



Slides courtesy: Prof Jonathon's slides on recursion



Slides courtesy: Prof Jonathon's slides on recursion

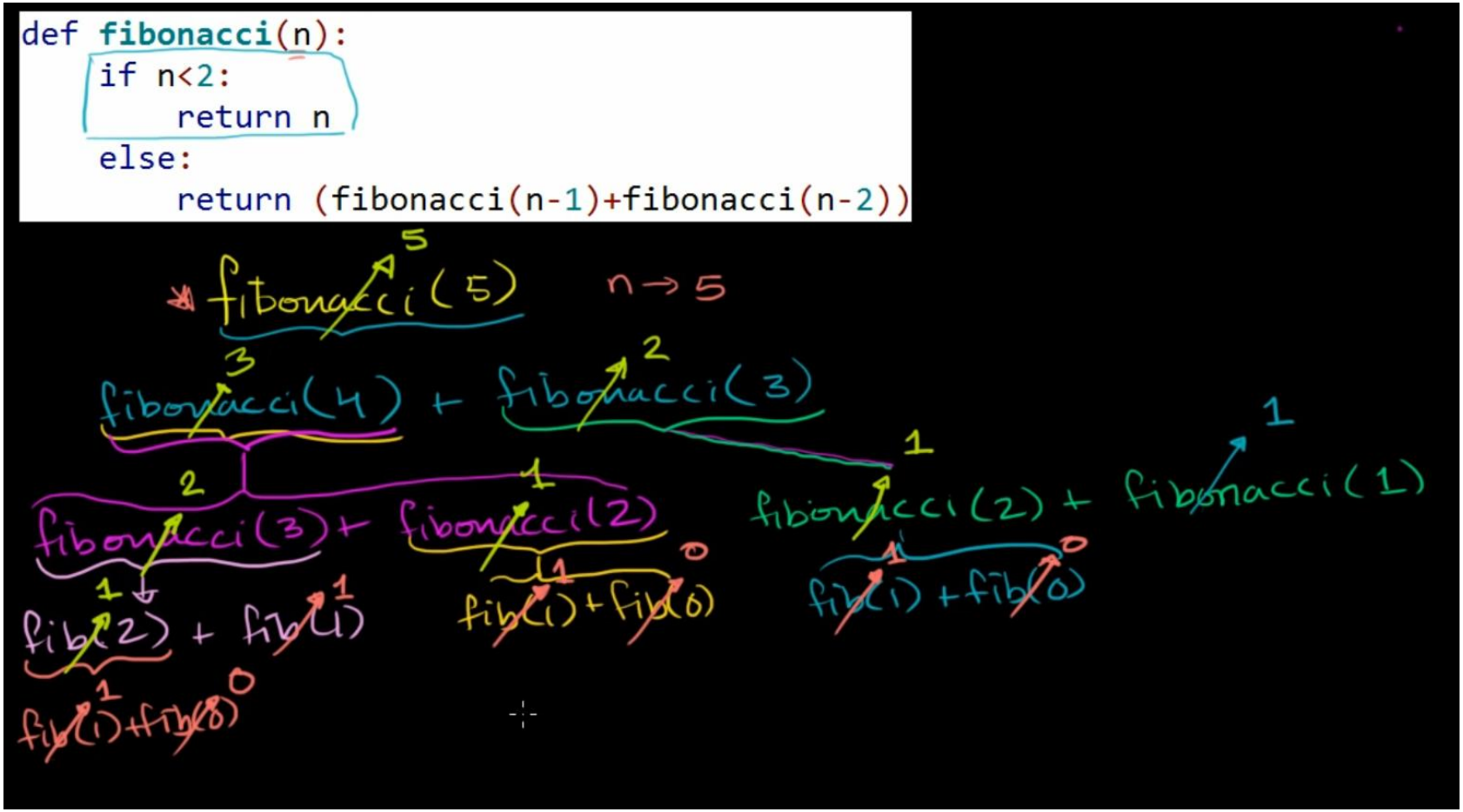
Two Other Classic Examples that Use Construct #1

- ***Fibonacci numbers*** are defined recursively, thus, coding them is quite easy, based on the original mathematical definition:
- $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$, *for all integers* $n > 1$.
- Here is a list:
  - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610,, ...

```
int fib(int n) {

  if (n <= 1)
    return n;
  else
    return fib(n-1) + fib(n-2);
}
```

Illustration of the Fibonacci recursion process using recursion tree



Example using construct 2

Construct 2

if (!(terminating condition)) {

Take a step closer to terminating condition

Call function RECURSIVELY on smaller subproblem

}

- Let's write a function to generate TIP chart:
- Write a function that takes in the lowest dollar value and highest dollar value on the chart. You know the TIP rate. The method header would look like the:

// The function prints out a chart with the appropriate tips

// for meals ranging from first_val to last_val number of

// dollars, for every whole dollar amount. Both parameters

// must be positive integers.

void Tip_Chart (int first_val, int last_val)

- Now, let's consider writing this function recursively.
- **First,** we need to determine a terminating condition.
  - if the lowest dollar amount is greater than the highest dollar amount, then the chart is empty and we are done.
- **Next,** we need to figure out how to break down our task into two steps: one that does part of the job, and secondly a recursive step.

Example using construct 2

- **Tip Chart:**
- What would be the first print out and what is next?
  - It is clear that the first value we have to print of the chart is associated with the lowest dollar value, since that must appear on the chart first.
  - Printing out this line of the tip chart can be the part of the job we accomplish.
- Now, are we left with a sub-problem of the same nature?
  - Yes! We simply now need to print a tip chart from the lowest dollar value+1 to the highest dollar value.

Example using construct 2

```
#define     TIP_RATE    0.15

// Pre-condition: Both parameters are integers with the first one being less than or equal to the second one.
// Post-condition: A tip chart will be printed out, with a row for each dollar amount ranging in between the value of the two parameters.
void Tip_Chart (int first_val, int last_val)
{

   if ( !(first_val > last_val) ) {
                        printf("On a meal of $%d", first_val);
                        printf(" you should tip $%lf\n", first_val*TIP_RATE);
                        Tip_Chart(first_val + 1, last_val);
   }
}

int main ()
{
   Tip_Chart(1,10);

}
```

```
On a meal of $1 you should tip $0.150000
On a meal of $2 you should tip $0.300000
On a meal of $3 you should tip $0.450000
On a meal of $4 you should tip $0.600000
On a meal of $5 you should tip $0.750000
On a meal of $6 you should tip $0.900000
On a meal of $7 you should tip $1.050000
On a meal of $8 you should tip $1.200000
On a meal of $9 you should tip $1.350000
On a meal of $10 you should tip $1.500000
```

Practice problem

- Write a recursive function that takes a string and the length of the string in parameters, then print the string in reverse order.
- The prototype should look like this:
- *void print_reverse(char string[], int n)*
- *If "hello" and 5 is passed, it should print "0lleh"*
- What position should be printed first?
  - string[n-1]
- What would be the breaking condition?
  - If n==1

Solving using construct 1

```
// Pre-condition: string is a string of length n or greater.
// Post-condition: Prints out the first n characters of string in reverse order.
void print_reverse(char string[], int n) {

    // Only one character to print, so print it!
    if (n == 1)
        printf("%c", string[0]);

    // Solve the problem recursively: print the last character, then reverse
    // the substring without that last character.
    else {
        printf("%c", string[n-1]);
        print_reverse(string, n-1);
    }
}
```

Solving using construct 2

```
void printReverse(char word[], int length)
{
        if (length > 0)
        {
                printf("%c", word[length-1]);
                printReverse(word, length-1);
        }
}
```

Practice Problem

- Write a recursive function that:
- Takes in two non-negative integer parameters
    - Returns the product of these parameters
        - But it does NOT use multiplication to get the answer
    - So if the parameters are 6 and 4
    - The answer would be 24
- Multiplication MxN is actully adding M for N times
- 6*4 = adding 6, 4 times.
- So, 6*4 = 6 + 6 + 6 + 6
- Now write the function.

Practice problem-solution

```
// Precondition: Both parameters are non-negative integers.
// Postcondition: The product of the two parameters is returned.
int Multiply(int first, int second)
{
        if (( second == 0 ) || ( first = 0 ))
                return 0;
        else
                return (first + Multiply(first, second-1));
}
```

How can you modify it to work for negative number?

```
int Multiply (int first, int second) {
    if ( second < 0 )
        return -Multiply(first,-second);
    else if ( first < 0 )
        return -Multiply(-first, second);
    else if (( second == 0 ) || ( first = 0 ))
        return 0;
    else
        return (first +Multiply(first, second-1));
}
```

-Example: if you calculate 4 * -3, the program converts the result to negative and make -3 to +3 and then use the usual recursion in the else statement

- Example: if you calculate -4 * 3, the program converts the result to negative and make -4 to +3 and then use the usual recursion in the else statement
- Example: if you calculate -4 * -3, the program converts the result to negative and make -3 to +3
    - In the next call, it converts the result to negative again (which makes the result positive) and converts -4 to +4. Then it uses the usual recursion in the else statement

More exercises

- **1) Write a recursive function that determines the nth Lucas number. Here is the definition of the Lucas numbers:**

- **$L_1 = 1$, $L_2 = 3$, $L_n = L_{n-1} + L_{n-2}$, for all integers $n > 2$.**
-
- **// Pre-condition: n > 0**
- **// Post-condition: Returns the nth Lucas number, L_n.**

int Lucas(int n);

n^{th} Lucas number

```
// n'th Lucas number
#include <stdio.h>

// recursive function
int lucas(int n)
{
   // Base cases
   if (n == 1)
      return 1;
   else if (n == 2)
      return 3;
   else
      return lucas(n - 1) + lucas(n - 2);
}

// Driver Code
int main()
{
   int n = 9;
   printf("%d", lucas(n));
   return 0;
}
```

Try this

- How can you use recursion to convert and print Decimal to Binary?
- Remember the steps: division and reminder technique.
- The reminders should be printed from bottom to top.
- Next slide is copied from Number system slide to remind you about the steps

- Converting from base 10 to a new base *(division- remainder technique)*

$117_{10} = ?_2$

Step	Operation	Result (n/base)	Remainder (n%base)
Step 1	117 / 2	58	1
Step 2	58 / 2	29	0
Step 3	29/ 2	14	1
Step 4	14 / 2	7	0
Step 5	7 / 2	3	1
Step 6	3/2	1	1
	Read reverse order 1110101_2		



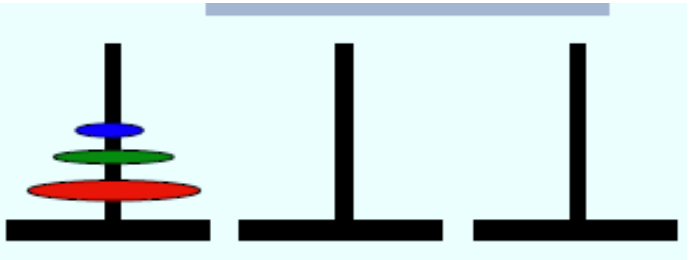
Decimal to Binary

```
void dectobin(int n) {

    if (n < 2)
        printf("%d", n);

    else {
        dectobin(n/2);
        printf("%d", n%2);
    }
}
```

Towers of Hanoi



- **<u>Problem:</u>**
- we have:
- three rods (or tower)
- n disks in the left most tower. The disks are arranged with the largest diameter disks at the bottom

- Some monk has the daunting task of moving disks from one tower to another tower
  - Often defined as moving from Tower #1 to Tower #3
    - Tower #2 is just an intermediate pole
  - He **can only move ONE disk at a time**
  - And he MUST follow the rule of **NEVER putting a bigger disk on top of a smaller disk**

Towers of Hanoi

- Steps:
- Take an example for 2 disks :
- Let rod 1 = 'A', rod 2 = 'B', rod 3 = 'C'.

- Step 1 : Shift first disk from 'A' to 'B'.
- Step 2 : Shift second disk from 'A' to 'C'.
- Step 3 : Shift first disk from 'B' to 'C'.

- The pattern here is :
- Shift 'n-1' disks from 'A' to 'B'.
- Shift last disk from 'A' to 'C'.
- Shift 'n-1' disks from 'B' to 'C'.

illustration for 3 disks :

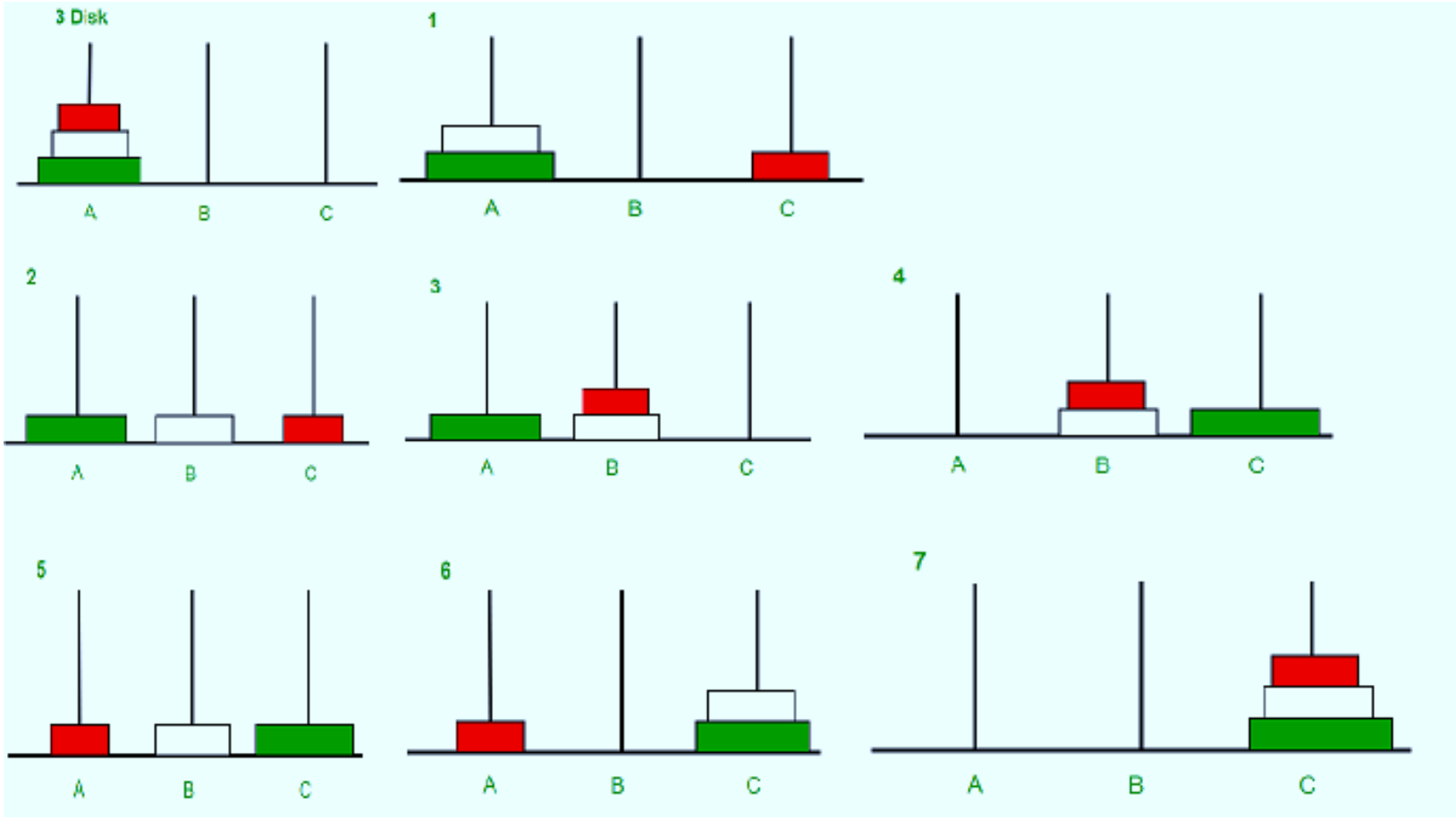
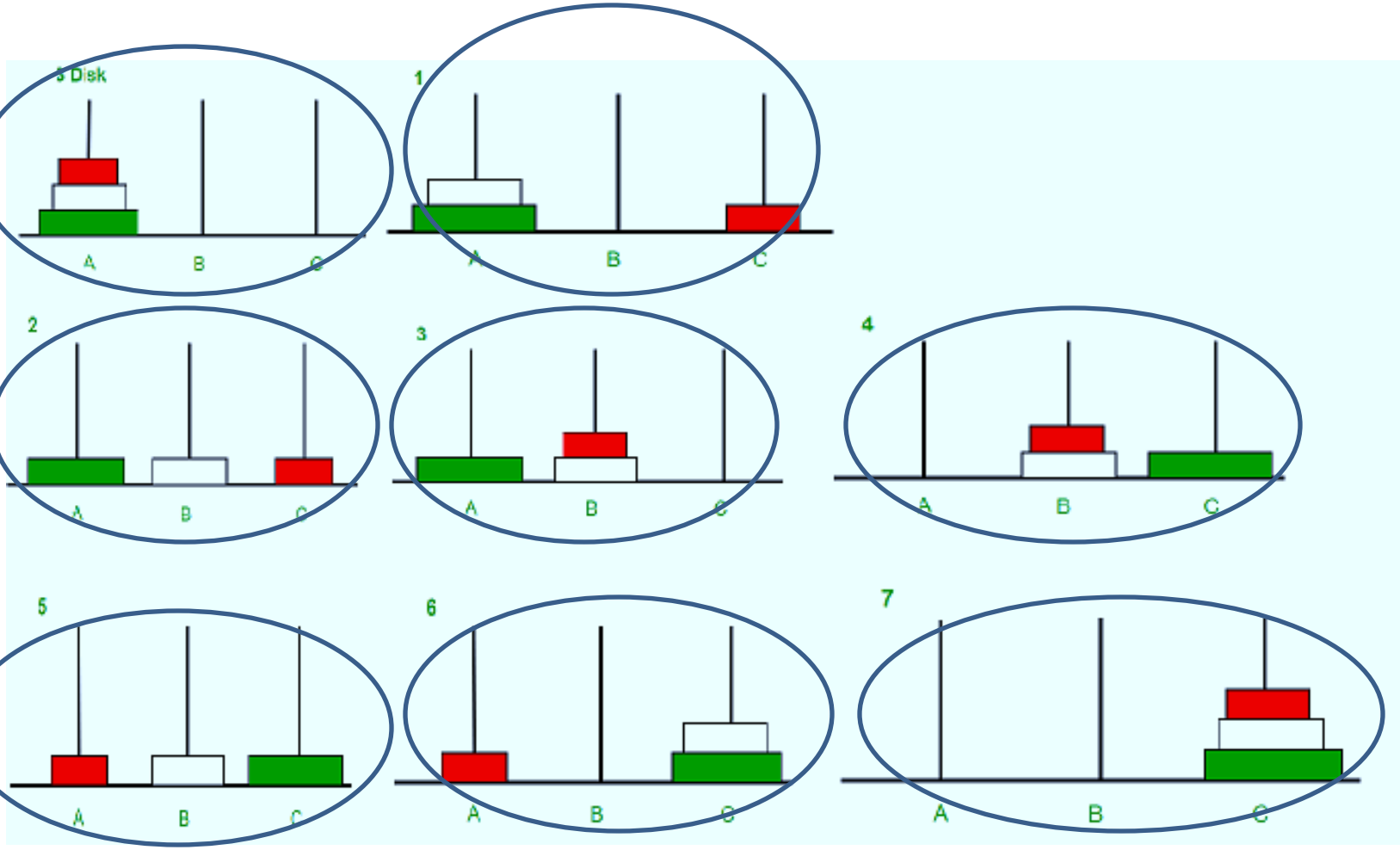


illustration for 3 disks :



Towers of Hanoi

- Input : 2

Output :

Disk 1 moved from A to B
Disk 2 moved from A to C
Disk 1 moved from B to C

- Input : 3

Output :

Disk 1 moved from A to C
Disk 2 moved from A to B
Disk 1 moved from C to B
Disk 3 moved from A to C
Disk 1 moved from B to A
Disk 2 moved from B to C
Disk 1 moved from A to C

The pattern here is :
Shift 'n-1' disks from 'A' to 'B'.
Shift last disk from 'A' to 'C'.
Shift 'n-1' disks from 'B' to 'C'.

Towers of Hanoi

```
#include <stdio.h>

void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod)
{
    if (n == 1)
    {
        printf("\n Move disk 1 from rod %c to rod %c", from_rod, to_rod);
        return;
    }
    towerOfHanoi(n-1, from_rod, aux_rod, to_rod);
    printf("\n Move disk %d from rod %c to rod %c", n, from_rod, to_rod);
    towerOfHanoi(n-1, aux_rod, to_rod, from_rod);
}

int main()
{
    int n = 4; // Number of disks
    towerOfHanoi(n, 'A', 'C', 'B');  // A, B and C are names of rods
    return 0;
}
```

Output:

```
Move disk 1 from rod A to rod B
Move disk 2 from rod A to rod C
Move disk 1 from rod B to rod C
Move disk 3 from rod A to rod B
Move disk 1 from rod C to rod A
Move disk 2 from rod C to rod B
Move disk 1 from rod A to rod B
Move disk 4 from rod A to rod C
Move disk 1 from rod B to rod C
Move disk 2 from rod B to rod A
Move disk 1 from rod C to rod A
Move disk 3 from rod B to rod C
Move disk 1 from rod A to rod B
Move disk 2 from rod A to rod C
Move disk 1 from rod B to rod C
```

Towers of Hanoi

For n disks, total $2^n - 1$ moves are required.

eg: For 4 disks $2^4 - 1 = 15$ moves are required.

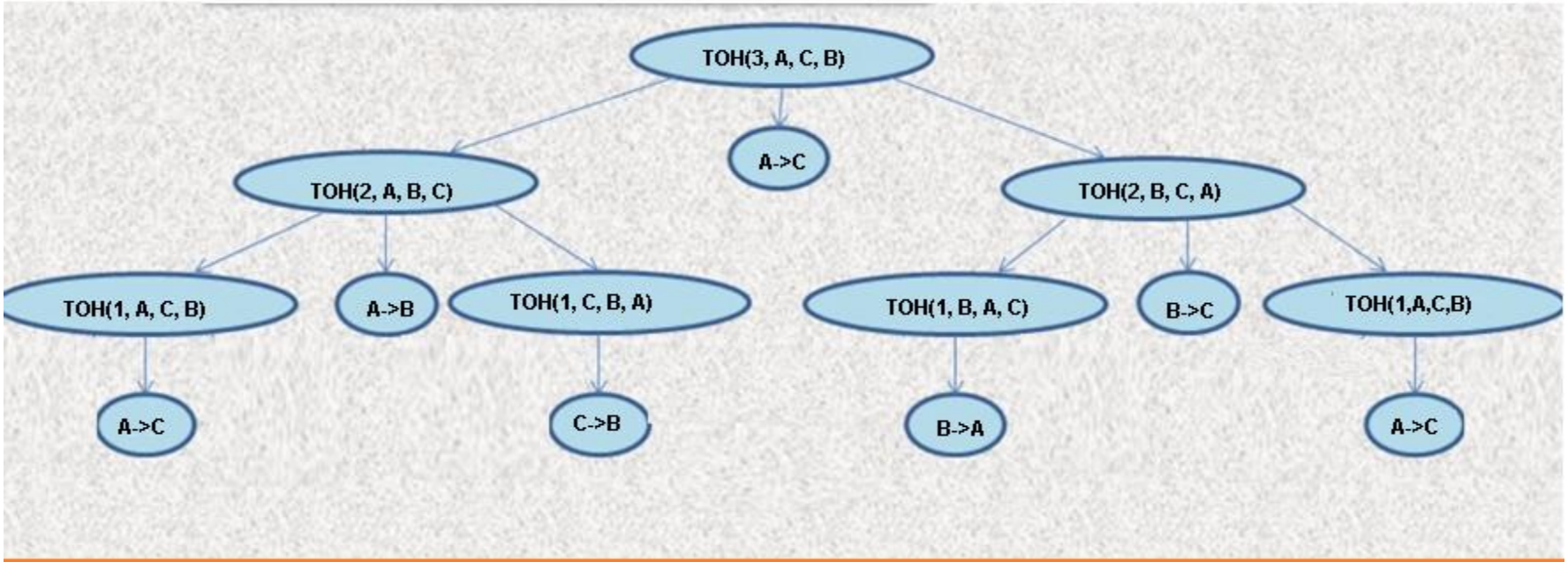
For n disks, total 2^n-1 function calls are made.

eg: For 4 disks 2^4-1 = 8 function calls are made.

Illustration of the recursion

```
void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod)
{
   if (n == 1)
   {
      printf("\n Move disk 1 from rod %c to rod %c", from_rod, to_rod);
      return;
   }
   towerOfHanoi(n-1, from_rod, aux_rod, to_rod);
   printf("\n Move disk %d from rod %c to rod %c", n, from_rod, to_rod);
   towerOfHanoi(n-1, aux_rod, to_rod, from_rod);
}

int main()
{
   int n = 4; // Number of disks
   towerOfHanoi(n, 'A', 'C', 'B');  // A, B and C are names of rods
   return 0;
}
```



Fast Exponent

```
int Power(int base, int exponent) {

    if ( exponent == 0 )
        return 1;
    else
        return (base*Power(base, exponent-1));
}
```

- Our recursive power function works just fine
- But, it becomes **VERY slow** for large exponents
  - If the exponent was 10,000, that would be 9,999 multiplications!
- How can we do better?
  - Remeber the laws of exponents:
    - **$2^8 = 2^4 * 2^4$**
    - **Now $2^4 = 2^2 * 2^2$**
    - **And $2^2 = 2 * (2)$**
  - So: 2*(2) = 4, 4*(4) = 16, 16*(16) = 256 = 2^8
  - So we've calculated 2^8 using only three multiplications
  - MUCH better than 7 multiplications

Fast Exponent

- So, we see that $b^n = b^{n/2} * b^{n/2}$
- **It will be log n number of multiplication (much better than n multiplication)**
- **But can you guess what problem we have here: $b^n = b^{n/2} * b^{n/2}$**
- **It works when n is even. How about if n is not even?**
- See: $2^9 = 2^4 * 2^4 * 2$
- *So, we need to handle this case. We can say:*

$$a^n = \begin{cases} a^{n/2}(a^{n/2}) & \text{if n is even} \\ a^{n/2}(a^{n/2})(a) & \text{if n is odd} \end{cases}$$

- Also note that this method relies on integer division.
  - **if n is 9, then n/2 = 4**
- **So, can you write the code?**

Fast Exponent

```
int powerB(int base, int exp)
{
        if (exp == 0)
                return 1;
        if (exp == 1)
                return base;
        if (exp%2 == 0) //if exp is even
                return powerB(base*base, exp/2);
        else //if exp is odd
                return powerB(base*base, exp/2) *base;
}

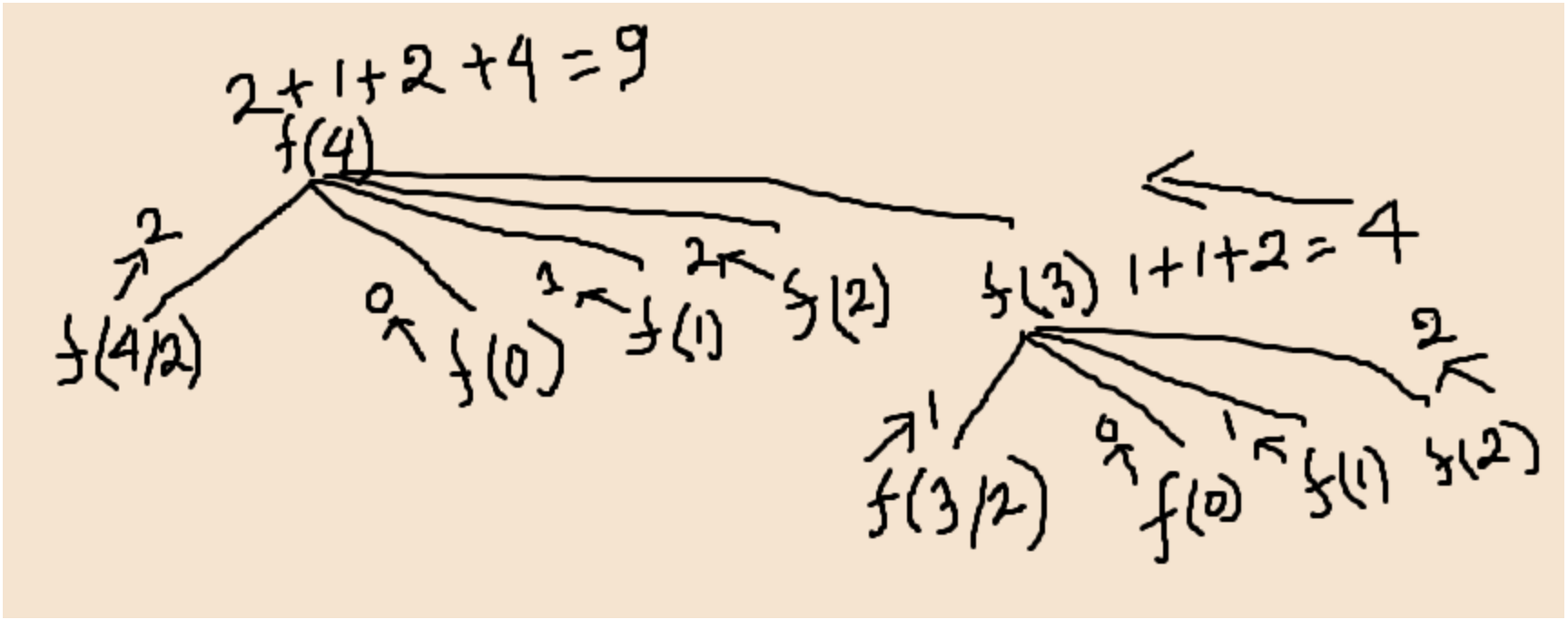

int main()
{
   printf("%d", powerB(2,5));

   return 0;
}
```

Let us try to find the result of f(4) for the following recursive function using recursion tree

```
int f(int n)
{
    int ans;
    int i;
    if(n<3)
        return n;
    ans = f(n/2);
    for(i=0; i<n; i++)
        ans += f(i);
    return ans;
}
```

```
int f(int n)
{
    int ans;
    int i;
    if(n<3)
        return n;
    ans = f(n/2);
    for(i=0; i<n; i++)
        ans += f(i);
    return ans;
}
```



More Recursion

- We have two more important recursion examples
  - Permutation
  - Flood Fill
  - Both of them are discussed in another slide PDF

References

- Tower of Hanoi: https://www.geeksforgeeks.org/c-program-for-tower-of-hanoi/
- Arup Guha's notes:
  - http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/RecursionIntro.doc
  - http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/RecursionTowers.doc
  - http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/RecursionFastExp.doc
  - http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/RecursionFloodFill.doc
  - http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/lec/RecursionPermutations.doc
- ***Many example functions in one c code:*** http://www.cs.ucf.edu/~dmarino/ucf/transparency/cop3502/sampleprogs/recursion.c